

Web Application Security Assessment

OWASP Juice Shop | Portfolio edition

Arica Bhuiyan | Based on coursework submitted in December 2025

Purpose and scope

This report presents a critical review of a university lab assessment of OWASP Juice Shop, a deliberately vulnerable training application. The original work used Burp Suite, browser tools and manual request modification against a local lab at localhost:3000. This edition shows the testing approach, evidence limits and recommended controls.

How to interpret this edition

The original application has not been rerun. Results below are explicitly attributed to the coursework record. Descriptions that originally treated limited evidence as conclusive have been corrected. Unverified findings remain hypotheses rather than confirmed vulnerabilities.

Privacy and evidence

The student number, original request screenshots, credentials and session tokens are not reproduced. This is a text-only public portfolio edition. The original assessment contains screenshots, but these were not independently validated as proof of each technical conclusion. Sanitised screenshots can be added after review.

Attribution

Lab assessment and original report: Arica Bhuiyan. Training platform and challenge material: OWASP Juice Shop. Editorial corrections and portfolio formatting: AI-assisted. This report assesses a third-party training application; it does not claim authorship of Juice Shop.

Assessment overview

Eight areas were investigated in the coursework. The key portfolio skill is connecting a test to defensible evidence, then recommending a control that addresses the demonstrated cause.

Area	Evidence status
Authentication / Login Admin	Not verified from the recorded procedure
Feedback persistence / Stored XSS	Persistence reported; script execution not demonstrated
Access to the administration route	Administration page access reported; API privileges need verification
Basket quantity manipulation	Quantity change reported; business-rule violation needs definition
Viewing another user's basket	Cross-user access reported; strong candidate for object-level authorisation failure
JSONP callback on the profile endpoint	Callback response reported; cross-origin disclosure not established
Passwords in change-password URL	Sensitive values in query parameters reported
Unexpected error response	Error handling challenge reported; disclosure impact not established

Risk scoring

The original DREAD totals are retained on the finding pages for traceability. The original report applied inconsistent severity labels to similar totals and did not define thresholds. This edition removes those labels. The totals are student estimates, not validated risk ratings, and should be reassessed after the evidence gaps are resolved.

01 | Authentication / Login Admin

Evidence status: Not verified from the recorded procedure

Original DREAD estimate: 45/50. Historical score; not reassessed or severity-labelled.

Recorded observation

The coursework describes forwarding a login request with an ordinary email and weak password, then observing a Login Admin challenge banner. These observations are reported from the original assessment; the lab has not been rerun for this edition.

Corrected interpretation

A successful login with valid credentials is not an authentication bypass. A browser password-save prompt does not prove administrator access. Seeing a password in a request inspected by an authorised proxy does not establish how the server stores passwords. The official Login Admin tutorial demonstrates SQL injection, but the recorded procedure does not document that payload.

Potential impact and limits

An actual authentication bypass could expose privileged functionality. Administrator access and its extent must be demonstrated before assigning that impact to this test.

Evidence needed

Reproduce the test in the authorised lab. Capture the relevant request, response, account identity and a protected operation. Determine whether injection, weak credentials or another mechanism caused the result.

Recommended controls

Use parameterised queries where injection is the cause. Enforce server-side authentication and authorisation; use appropriate password hashing, throttling and MFA for privileged accounts.

02 | Feedback persistence / Stored XSS

Evidence status: Persistence reported; script execution not demonstrated

Original DREAD estimate: 42/50. Historical score; not reassessed or severity-labelled.

Recorded observation

The documented request stores the benign comment "Great products to buy!!", which is subsequently displayed. The report also mentions trying script payloads, but does not provide a complete recorded execution result for those attempts.

Corrected interpretation

Storing and displaying ordinary text is expected functionality. It does not prove missing output encoding or executable stored XSS. The result should be recorded as feedback persistence, with an XSS hypothesis requiring further evidence.

Potential impact and limits

If a stored payload executes in another user session, it could perform actions in that application context. Token theft depends on storage and browser protections; it should not be assumed.

Evidence needed

Record a harmless test payload, its storage request and later execution in a separate authorised test session. Identify the rendering context and distinguish stored, reflected and DOM-based behaviour.

Recommended controls

Use context-appropriate output encoding and safe rendering APIs. Sanitise HTML only where rich text is intentionally supported. Add Content Security Policy as defence in depth.

03 | Access to the administration route

Evidence status: Administration page access reported; API privileges need verification

Original DREAD estimate: 39/50. Historical score; not reassessed or severity-labelled.

Recorded observation

The assessment reports navigating to `/#/administration` while signed in as a standard user and receiving the Admin Section challenge banner.

Corrected interpretation

Access to a client-side route alone does not prove that every backend administrator operation is unprotected. Sensitive data retrieval or a protected action needs separate evidence, together with the role of the test account.

Potential impact and limits

Unauthorised exposure of administrator data would affect confidentiality. User deletion, role modification and complete takeover were not demonstrated by the described route test.

Evidence needed

Record the standard user identity and the API responses used by the page. Compare permitted and forbidden behaviour for standard and administrator accounts.

Recommended controls

Enforce role checks at protected API endpoints. Client-side route guards improve navigation but do not replace server-side access control. Return only data required by the authorised role.

04 | Basket quantity manipulation

Evidence status: Quantity change reported; business-rule violation needs definition

Original DREAD estimate: 40/50. Historical score; not reassessed or severity-labelled.

Recorded observation

The assessment reports changing the quantity in PUT /api/BasketItems/1 from 2 to 1000 and observing a basket total that increased with the quantity.

Corrected interpretation

An accepted quantity of 1000 is not automatically a vulnerability unless it violates stock or purchasing rules. Changing a quantity on an owned item is not itself IDOR. The reported increase in total price does not demonstrate buying more goods without paying.

Potential impact and limits

If stock or purchase limits can be bypassed, the behaviour could create order-processing or inventory problems. Financial loss and service exhaustion were not demonstrated.

Evidence needed

Define the expected quantity limits, inspect stock enforcement and verify checkout behaviour in the lab. Test ownership separately from numeric validation.

Recommended controls

Validate quantity type, lower bound, stock and purchase limits on the server. Calculate prices from trusted data. Authorise ownership of each basket item. Do not attempt to distinguish legitimate frontend clicks from crafted HTTP requests.

05 | Viewing another user's basket

Evidence status: Cross-user access reported; strong candidate for object-level authorisation failure

Original DREAD estimate: 41/50. Historical score; not reassessed or severity-labelled.

Recorded observation

The coursework reports changing GET /rest/basket/2 to /rest/basket/1 while remaining signed in as a different user, retrieving the other basket and receiving the View Basket banner.

Corrected interpretation

This is consistent with an object-level authorisation failure if the requesting account was not permitted to access that basket. Predictable identifiers make enumeration easier, but the missing permission check is the central issue.

Potential impact and limits

The demonstrated scope is another basket's contents. Modification of that basket, price manipulation or access to all accounts requires additional testing.

Evidence needed

Preserve sanitised request/response pairs and confirm account ownership before and after changing the ID. Check both read and write endpoints with two controlled test users.

Recommended controls

Check object ownership or an explicit permission on every request. Deny access where that check fails. Opaque IDs may reduce easy enumeration but are not a substitute for authorisation.

06 | JSONP callback on the profile endpoint

Evidence status: Callback response reported; cross-origin disclosure not established

Original DREAD estimate: 42/50. Historical score; not reassessed or severity-labelled.

Recorded observation

The coursework describes adding `callback=admin` to `/rest/user/whoami` and receiving a JavaScript-wrapped response with profile information.

Corrected interpretation

The name "admin" is a callback function name, not a role change. An authenticated endpoint returning the current user's own profile is not automatically a leak. A JSONP response alone does not establish that a different origin can obtain an authenticated victim's data. A script request cannot normally attach the bearer header used in the recorded proxy request.

Potential impact and limits

Cross-origin disclosure would matter if a browser can send the required credentials and expose sensitive data to another origin. That browser behaviour was not demonstrated in the written procedure.

Evidence needed

Test a separate controlled origin and document the credential mechanism, browser policy, actual request and whether private data becomes readable. Treat CORS and JSONP as different mechanisms.

Recommended controls

Remove unnecessary JSONP support from sensitive endpoints and return JSON with an appropriate content type. Authorise access, minimise response data and configure cross-origin access deliberately. Changing CORS alone does not fix JSONP.

07 | Passwords in change-password URL

Evidence status: Sensitive values in query parameters reported

Original DREAD estimate: 47/50. Historical score; not reassessed or severity-labelled.

Recorded observation

The assessment records

/rest/user/change-password?current=[REDACTED]&new=[REDACTED]&repeat=[REDACTED], with password values in the URL query. The original values and request screenshots are excluded here.

Corrected interpretation

Query parameters can be captured in application and proxy logs or monitoring systems. Browser-history exposure depends on whether the URL was navigated to; background API requests are not necessarily added to history. Plaintext visible in an authorised proxy does not prove plaintext transport when HTTPS is in use. This finding does not by itself establish broken access control.

Potential impact and limits

People or systems with access to logs containing these parameters could obtain credentials. The report does not demonstrate a separate attacker obtaining or replaying them.

Evidence needed

Confirm the request method, logging behaviour and access to resulting logs in the lab. Check password reauthentication, authorisation and protections appropriate to the session mechanism.

Recommended controls

Use a state-changing request with sensitive fields in the body over HTTPS. Redact secrets in logs, require appropriate reauthentication, validate the session and use CSRF protection where applicable. A body alone does not prevent leakage if bodies are logged.

08 | Unexpected error response

Evidence status: Error handling challenge reported; disclosure impact not established

Original DREAD estimate: 36/50. Historical score; not reassessed or severity-labelled.

Recorded observation

The coursework reports searching for "orange", receiving an unexpected error response and seeing an Error Handling challenge notification.

Corrected interpretation

The error body, response status and reproducible triggering conditions are not documented in sufficient detail to establish sensitive information disclosure or a denial-of-service vulnerability.

Potential impact and limits

An error may expose implementation details if its response contains them. A generic error or a single failed request does not establish that impact.

Evidence needed

Reproduce the request and preserve a sanitised response, including status and relevant error content. Check whether the failure is repeatable and whether sensitive details are returned.

Recommended controls

Validate inputs, handle exceptions centrally and send appropriate generic responses to users. Keep diagnostic detail in protected logs with secrets redacted.

Conclusion and next steps

This assessment demonstrates practical experience with HTTP inspection, request modification, authentication and authorisation testing, business-rule checks, and security reporting. The cross-user basket test and passwords-in-URL observation are useful reported findings. Other sections require stronger evidence before a vulnerability and its impact can be claimed.

Recommended follow-up

Reproduce each test in a fresh, authorised Juice Shop lab and record the application version. Use controlled accounts, retain sanitised request/response pairs, distinguish account roles, and capture the actual effect rather than relying only on a challenge banner. Reassess risk after confirmation.

Future development project

A small login application can demonstrate the defensive controls discussed here: password hashing, server-side sessions, ownership checks, CSRF protection, safe template rendering and carefully handled errors. This is planned follow-up work, not a completed part of this assessment.

References

OWASP Juice Shop official challenge guide: <https://help.owasp-juice.shop/>

Login Admin tutorial: <https://help.owasp-juice.shop/part3/tutorials.html>

OWASP Stored XSS testing guide: https://wstg.owasp.org/latest/4-Web_Application_Security_Testing/07-Injection/02-Stored_Cross_Site_Scripting/

OWASP Access Control Cheat Sheet:
https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html

OWASP XSS Prevention Cheat Sheet:
https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

Original university coursework report supplied by Arica Bhuiyan, December 2025.